BDT Lab – Experiment 6 – Support Doc

Adesh Patankar

Batch A

BDT LAB: 06

Problem Statement: Implement the various operations on Hive. (Create, Insert, Update). **Outcome:**

Students will be able to implement the various operations on Hive. (Create, Insert, Update)

Theory:

Hive is a data warehouse infrastructure tool to process structured data in Hadoop. It resides on top of Hadoop to summarize Big Data, and makes querying and analyzing easy.

Initially Hive was developed by Facebook, later the Apache Software Foundation took it up and developed it further as an open source under the name Apache Hive. It is used by different companies. For example, Amazon uses it in Amazon Elastic MapReduce. Hive is not

· A relational database

· A design for OnLine Transaction Processing (OLTP)

· A language for real-time queries and row-level updates

Features of Hive

· It stores schema in a database and processed data into HDFS.

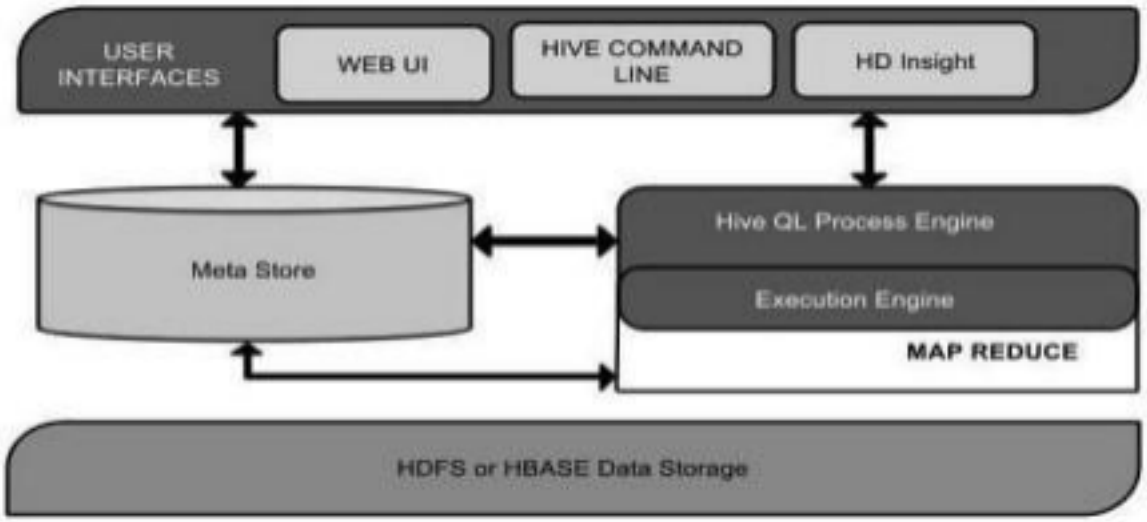
· It is designed for OLAP.

· It provides SQL type language for querying called HiveQL or HQL.

· It is familiar, fast, scalable, and extensible.

Architecture of Hive

The following component diagram depicts the architecture of Hive:



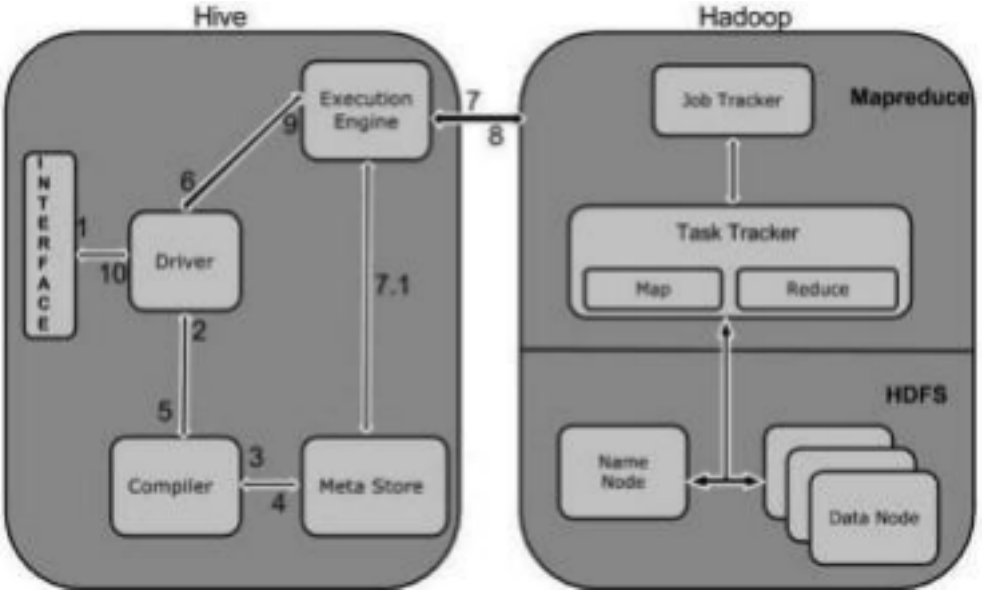
This component diagram contains different units. The following table describes each unit:

Unit Name	Operation

User Interface	Hive is a data warehouse infrastructure software that can create interaction between user and HDFS. The user interfaces that Hive supports are Hive Web UI, Hive command line, and Hive HD Insight (In Windows server).
Meta Store	Hive chooses respective database servers to store the schema or Metadata of tables, databases, columns in a table, their data types, and HDFS mapping.
HiveQL Process Engine	HiveQL is similar to SQL for querying on schema info on the Metastore. It is one of the replacements of traditional approach for MapReduce program. Instead of writing MapReduce program in Java, we can write a query for MapReduce job and process it.
Execution Engine	The conjunction part of HiveQL process Engine and MapReduce is Hive Execution Engine. Execution engine processes the query and generates results as same as MapReduce results. It uses the flavor of MapReduce.
HDFS or HBASE	Hadoop distributed file system or HBASE are the data storage techniques to store data into file system.

Working of Hive

The following diagram depicts the workflow between Hive and Hadoop.



The following table defines how Hive interacts with Hadoop framework:

Step No.	Operation

1	**Execute Query** The Hive interface such as Command Line or Web UI sends query to Driver (any database driver such as JDBC, ODBC, etc.) to execute.
2	**Get Plan** The driver takes the help of query compiler that parses the query to check the syntax and query plan or the requirement of query.
3	**Get Metadata** The compiler sends metadata request to Metastore (any database).
4	**Send Metadata** Metastore sends metadata as a response to the compiler.
5	**Send Plan** The compiler checks the requirement and resends the plan to the driver. Up to here, the parsing and compiling of a query is complete.
6	**Execute Plan** The driver sends the execute plan to the execution engine.
7	**Execute Job** Internally, the process of execution job is a MapReduce job. The execution engine sends the job to JobTracker, which is in Name node and it assigns this job to TaskTracker, which is in Data node. Here, the query executes MapReduce job.
7.1	**Metadata Ops** Meanwhile in execution, the execution engine can execute metadata operations with Metastore.

8	**Fetch Result** The execution engine receives the results from Data nodes.

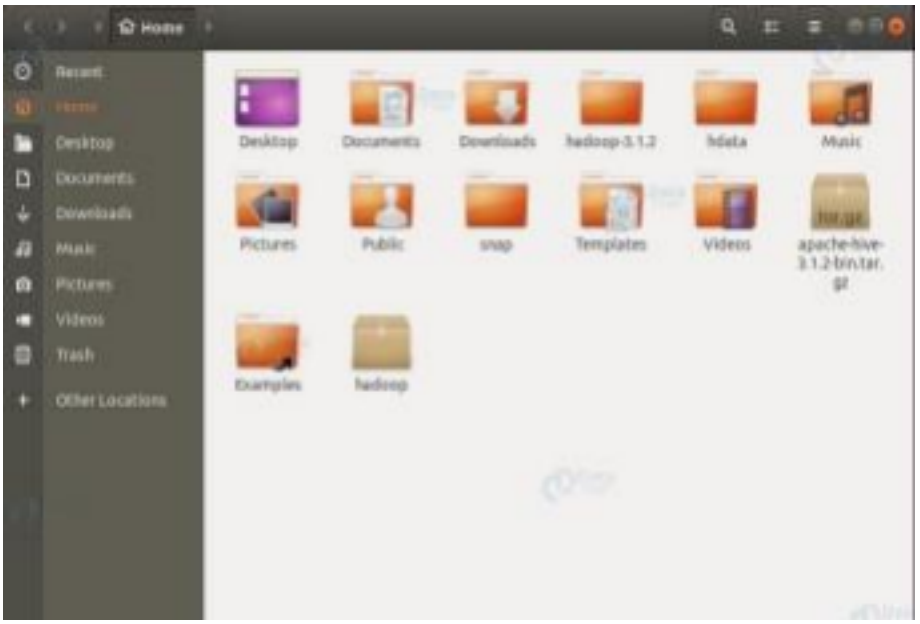
9	**Send Results** The execution engine sends those resultant values to the driver.
10	**Send Results** The driver sends the results to Hive Interfaces.

Hive - Installation

1. Download Hive

Step 1: First, download the Hive 3.1.2 from this link.

Step 2: Locate the apache-hive-3.1.2-bin.tar.gz file in your system.



Step 3: Extract this tar file using the below command:

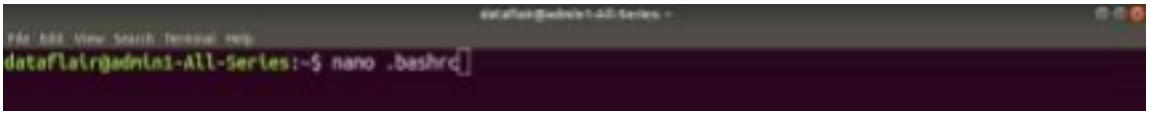
tar -xzf apache-hive-3.1.2-bin.tar.gz

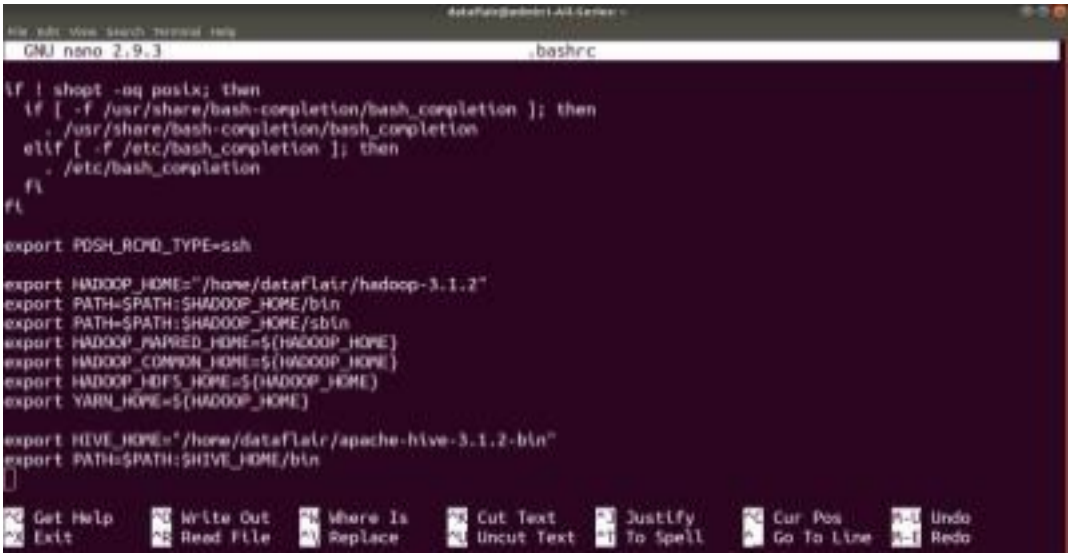




2. Configuring Hive files

Step 4: Now, we have to place the Hive PATH in .bashrc file. For this, open .bashrc file in the nano editor and add the following in the .bashrc file. export HIVE_HOME= “home/dataflair/apache-hive 3.1.2-bin” export PATH=$PATH:$HIVE_HOME/bin





Note: Here enter the correct name & version of your hive and correct path of your Hive File “home/dataflair/apache-hive-3.1.2-bin” this is the path of my Hive File and “apache-hive-3.1.2-bin” is the name of my hive file.

So please enter the correct path and name of your Hive file. After adding save this

file. Press CTRL+O and enter to save changes. Then press CTRL+D to exit the

editor.

Step 5: Open the core-site.xml file in the nano editor. The file is located in home/hadoop 3.1.2/etc/hadoop/ (Hadoop Configuration Directory).

Add the following configuration property in the core-site.xml file.

<configuration>

<property>

<name>hadoop.proxyuser.dataflair.groups</name>

<value>*</value>

</property>

<property>

<name>hadoop.proxyuser.dataflair.hosts</name>
<value>*</value>

</property>

<property>

<name>hadoop.proxyuser.server.hosts</name>

<value>*</value>

</property>

<property>

<name>hadoop.proxyuser.server.groups</name>

<value>*</value>

</property>

</configuration>

Press CTRL+O and enter to save changes. Then press CTRL+D to exit

the editor.

Step 6: Make a directory ‘tmp’ in HDFS using the below command:

hadoop fs -mkdir /tmp

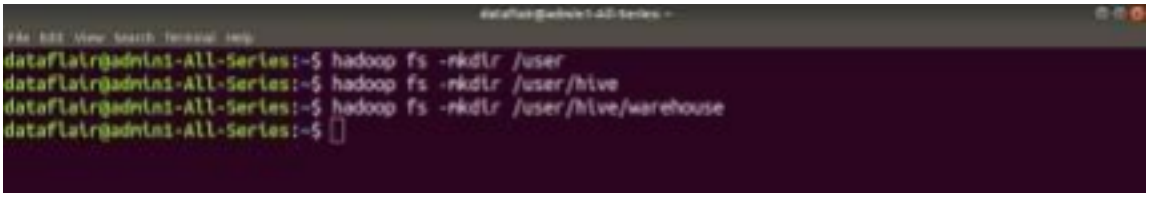


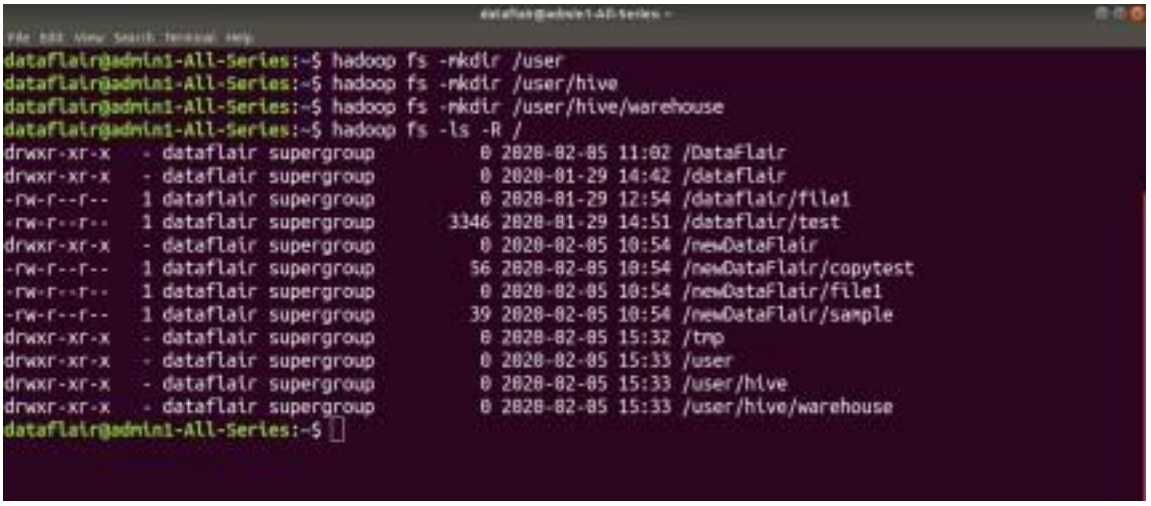
Step 6: Use the below commands to create a directory ‘warehouse’ inside ‘hive’ directory, which resides in ‘user’ directory. The warehouse is the location to store data or tables related to Hive.

hadoop fs -mkdir /user hadoop fs -

mkdir /user/hive hadoop fs -mkdir

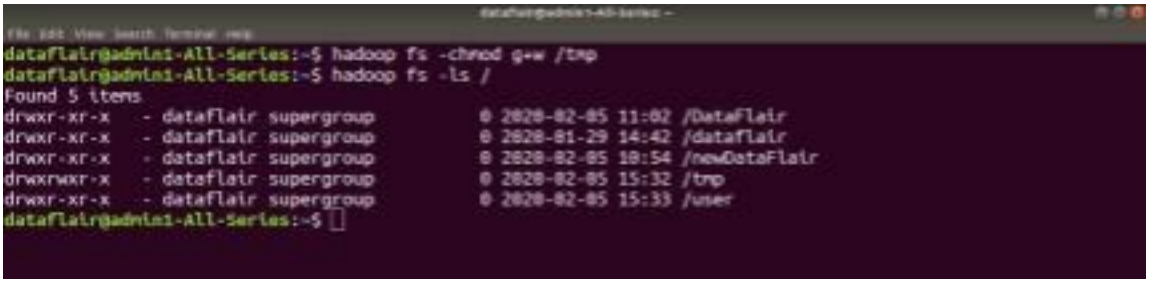
/user/hive/warehouse





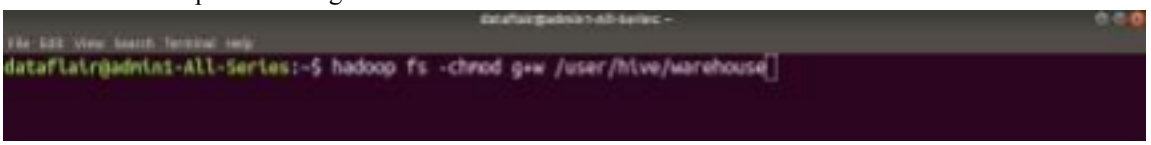
Step 7: Give the write permission to the members of the 'tmp' file group using command:

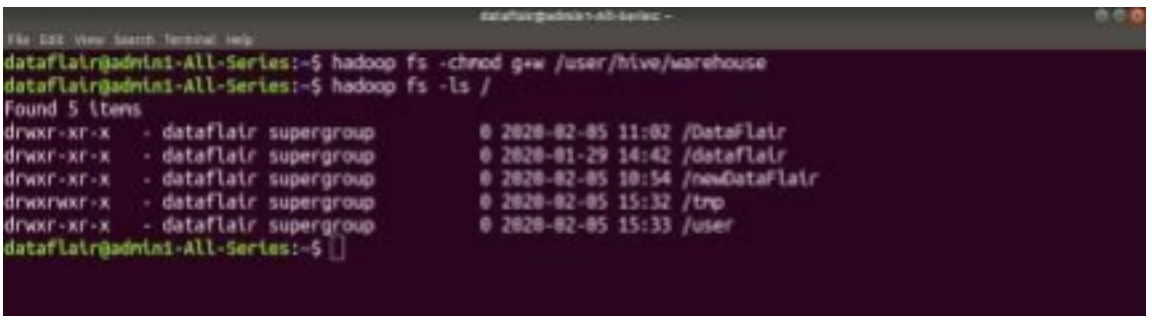




Step 8: Now give write permission to the warehouse directory using the

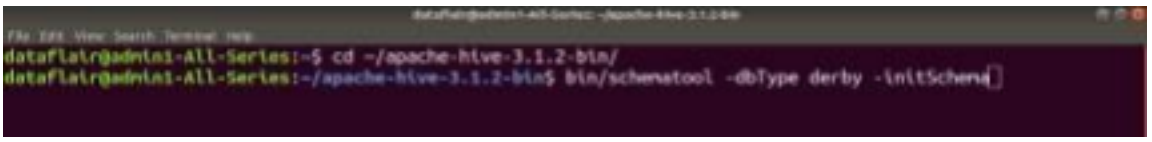
command: hadoop fs -chmod g+w /user/hive/warehouse





3. Initialize Derby database

Step 9: Hive by default uses Derby database. Use the below command to initialize the Derby database.
bin/schematool -dbType derby -initSchema



4. Launching Hive

Step 10: Now start the HiveServer2 using the below command:

bin/hiveserver2

[For this first move to the ~/apache-hive-3.1.2-bin/]

Step 11: On the different tab, type the below command to launch the beeline command shell.
bin/beeline -n dataflair -u jdbc:hive2://localhost:10000

Congratulations!! We have successfully installed Hive 3.1.2 on Ubuntu.

Now, you can type SQL queries in the Beeline command shell to interact with the Hive

system. **5. Verifying Hive Installation**

For example, in the below image, I am using show databases query to list out the database in the Hive warehouse.

We have successfully installed Apache Hive 3.1.2 on Hadoop 3.1.2 on Ubuntu.

Hive – Create Database Examples

Using CREATE DATABASE statement you can create a new Database in Hive, like any other RDBMS Databases, the Hive database is a namespace to store the tables.

In this article, I will explain how to create a database, its syntax, and usage with examples in hive shell, Java and Scala languages.

Hive by default contains a default database, you can get this using SHOW DATABASES; statement. Note: when you running SQL statements on Hive or beeline shell terminal, the statement should end with a semicolon.

Hive Create Database Syntax

In Hive, CREATE DATABASE statement is used to create a Database, this takes an optional clause IF NOT EXISTS, using this option, it creates only when database not already exists.

CREATE DATABASE [IF NOT EXISTS] <database_name>

Note: Creating a database with already existing name in a database returns

an error **Create Database from Hive Beeline shell**

jdbc:hive2://> CREATE DATABASE emp;

OK

Time taken: 0.929 seconds

jdbc:hive2://>

jdbc:hive2://> CREATE DATABASE emp;

FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.ql.exec.DDLTask. Database emp already exists
If you wanted to check if the database already exists before creating, use IF NOT

EXISTS clause. dbc:hive2://> CREATE DATABASE IF NOT EXISTS emp;

OK

Time taken: 0.063 seconds

dbc:hive2://>

Hive by default creates a directory with a database name .db extension (for example emp.db) under its warehouse directory. By default warehouse directory located is /user/hive/warehouse on hdfs.

rabha@prabha:~$ hdfs dfs -ls /user/hive/warehouse
Found 1 items
drwxr-xr-x - prabha supergroup 0 2020-10-03 21:30 /user/hive/warehouse/emp.db

You can change the location of the database where to create by using any of the below commands

dbc:hive2://>CREATE DATABASE temp LOCATION '/apps/project/hive/warehouse'; You can also change the default location using `hive.metastore.warehouse.dir` dbc:hive2://>set hive.metastore.warehouse.dir='/apps/project/hive/warehouse' jdbc:hive2://>CREATE

DATABASE temp;

Show Database

In order to show the existing databases on Hive, use the SHOW DATABASES command. dbc:hive2://>> SHOW DATABASES;

```
OK
default
emp
Time taken: 0.059 seconds, Fetched: 2 row(s)
```

Hive Create Table Syntax & Usage with Examples

Hive CREATE TABLE statement is used to create a table, it is similar to creating a table in RDBMS using SQL syntax, additionally, Hive has many more features to work with files.

1. Hive Create Table Syntax

By using CREATE TABLE statement you can create a table in Hive, It is similar to SQL and CREATE TABLE statement takes multiple optional clauses,

```
CREATE [TEMPORARY] [EXTERNAL] TABLE [IF NOT EXISTS] [db_name.]
table_name
[(col_name data_type [column_constraint] [COMMENT col_comment], ...)]
[PARTITIONED BY (col_name data_type [COMMENT 'col_comment'],
...)] [CLUSTERED BY (col_name, col_name,.....]
 COMMENT table_comment]
[ROW FORMAT row_format]
[FIELDS TERMINATED BY char
[LINES TERMINATED BY char]
 [LOCATION 'hdfs_path']
[STORED AS file_format]
```

In Hive, table can be created with or without the database, If you wanted to create in a database, specify database name qualifier.

Note: By default, the Hive tables are created in the warehouse directory, the location is specified in value for the key hive.metastore.warehouse.dir on the config file $HIVE_HOME/conf/hive-site.xml, by default, the warehouse location would be /user/hive/warehouse.

On this warehouse location, Hive creates a directory for every database you create and a subdirectory for every table, table data is stored in the table directory.

1.1 Create Table optional clauses

Below I have explained the most used optional clauses with examples.

IF NOT EXISTS – You can use IF NOT EXISTS to avoid the error in case the table is already present. Hive checks if the requesting table already presents,

`EXTERNAL` – Used to create external table
`TEMPORARY` – Used to create temporary t able.
`ROW FORMAT` – Specifies the format of the row.
`FIELDS TERMINATED BY` – By default Hive use **^A** field separator, To load a file that has a custom field separator like comma, pipe, tab use this option. `PARTITION BY` – Used to create partition data. Using this improves performance. `CLUSTERED BY` – Dividing the data into a specific number for buckets. `LOCATION` – You can specify the custom location where to store the data on HDFS.

2. Hive Create Table Example

2.1 Create Table

Here is a simple Hive example that creates an **employee** table in **emp** database with **id**, **name**, **age** and **gender** columns. we have also provided comment to the table.

```
CREATE TABLE IF NOT EXISTS
 emp.employee ( id int, name string, age int
 gender string )
 COMMENT 'Employee Table'
 ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
```

Note that the latest versions of Hive don't support the CHAR datatype hence used string for gender.

By default, Hive uses **^A** as a field separator while loading a file. but, I have a comma-separated file to load the data into this table hence, I've used ROW FORMAT DELIMITED FIELDS TERMINATED BY optional clause to specify the custom delimiter.

2.2 Describe Table

Use DESCRIBE command to describe the table.

```
dbc:hive2://127.0.0.1:10000> DESCRIBE emp.employee;
```

2.3 Load File into table

Let's see how to load a data file into the Hive table we just created.

Create a data file (for our example, I am creating a file with comma-separated fields)

Upload the data file (data.txt) to HDFS. Note you can also load the data from LOCAL without uploading to HDFS.

Now use the Hive LOAD command to load the file into the table.

```
LOAD DATA INPATH '/user/hive/data/data.txt' INTO TABLE emp.employee;
```

Note that after loading the data, the source file is not present in the location, and you can check the loaded file at HDFS.

```
hdfs dfs -ls /user/hive/warehouse/emp.db/employee/
-rw-r--r-- 1 hive supergroup 52 2020-10-09 19:29
/user/hive/warehouse/emp.db/employee/data.txt
```

2.4 Select table

Use SELECT command to select the get the data from a table.

2.5 Insert Date into Table

Like SQL, you can also use INSERT INTO to insert rows into Hive table.

```
INSERT INTO emp.employee values(7,'scott',23,'M');
INSERT INTO emp.employee values(8,'raman',50,'M');
```

3. Hive Table Types

3.1 Internal or Managed Table

By default, Hive creates an Internal table also known as the Managed table, In the managed table, Hive owns the data/files on the table meaning any data you insert or load files to the table are managed by the Hive process when you drop the table the underlying data or files are also get deleted.

Examples I have used above sections are Managed tables.

Let's see another example on how to LOAD file from LOCAL file system. LOAD

```
DATA LOCAL INPATH '/home/hive/data.txt' INTO TABLE emp.employee;
```

Unlike loading from HDFS, source file from LOCAL file system won't be

removed. **3.2 External Table**

Using EXTERNAL option you can create an external table, Hive doesn't manage the external table, when you drop an external table, only table metadata from Metastore will be removed but the underlying files will not be removed and still they can be accessed via HDFS commands, Pig, Spark or any other Hadoop compatible tools.

```
CREATE EXTERNAL TABLE
emp.employee_external ( id int, name string, age int
gender string
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ','
LOCATION '/user/hive/data/employee_external'
```

Now, let's LOAD the data from local file system.

```
LOAD DATA LOCAL INPATH '/home/hive/data.txt' INTO TABLE
emp.employee_external;
```

This will create a folder `/user/hive/data/employee_external/` on HDFS and `data.txt` .

check if the file created at Hive external location on HDFS .

hdfs dfs -ls /user/hive/data/employee_external/

-rw-r--r-- 1 hive supergroup 52 2020-10-09 20:29 /user/hive/data/employee_external/data.txt

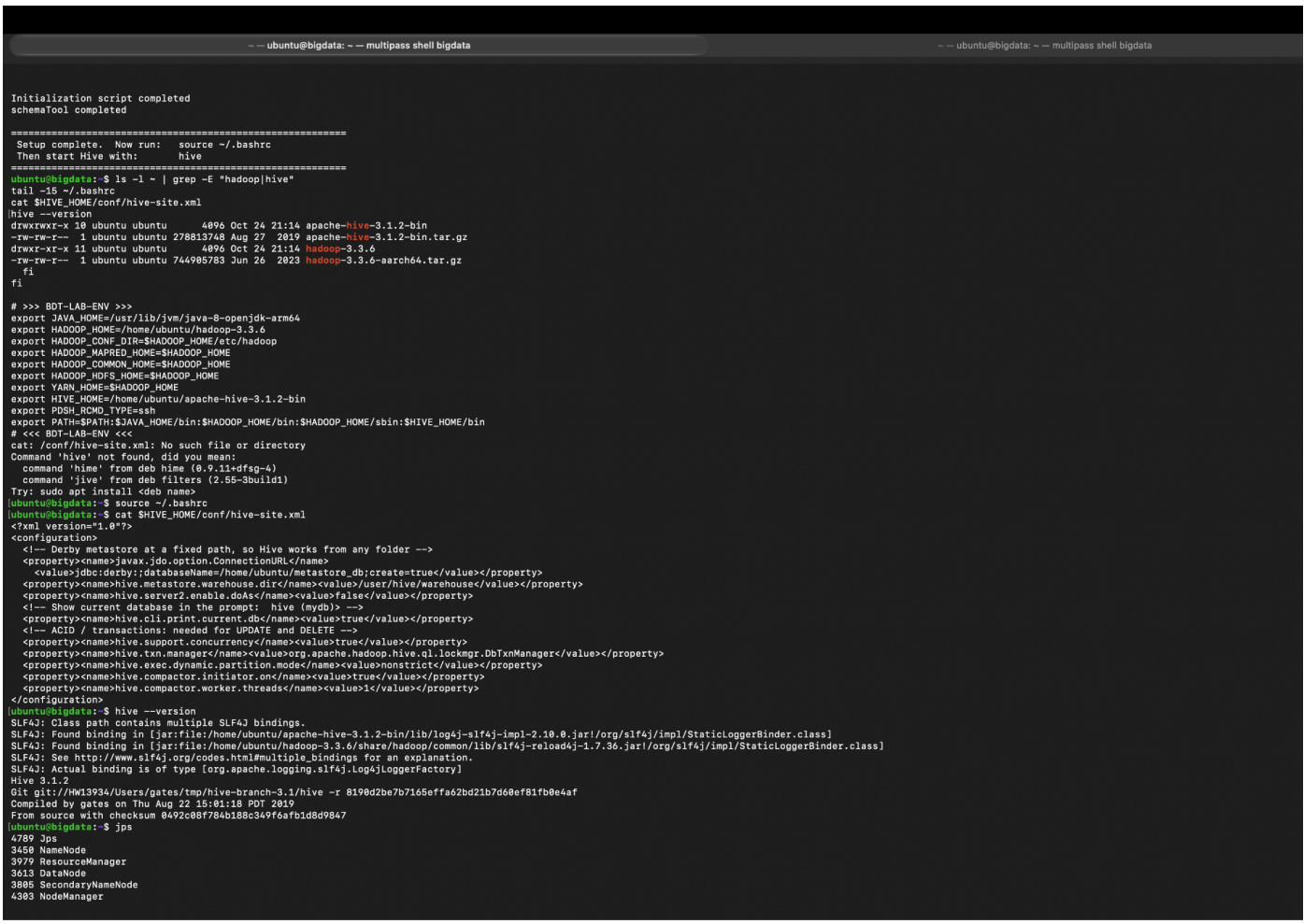
creates a file

Let's

As explained earlier dropping an external table from Hive will just remove the metadata from Metastore but not the actual file on HDFS.

Let's see this in action by dropping the table **emp.employee_external** using `DROP TABLE emp.employee_external` command and check if the file still exists by running above `hdfs -ls` command.

Results:



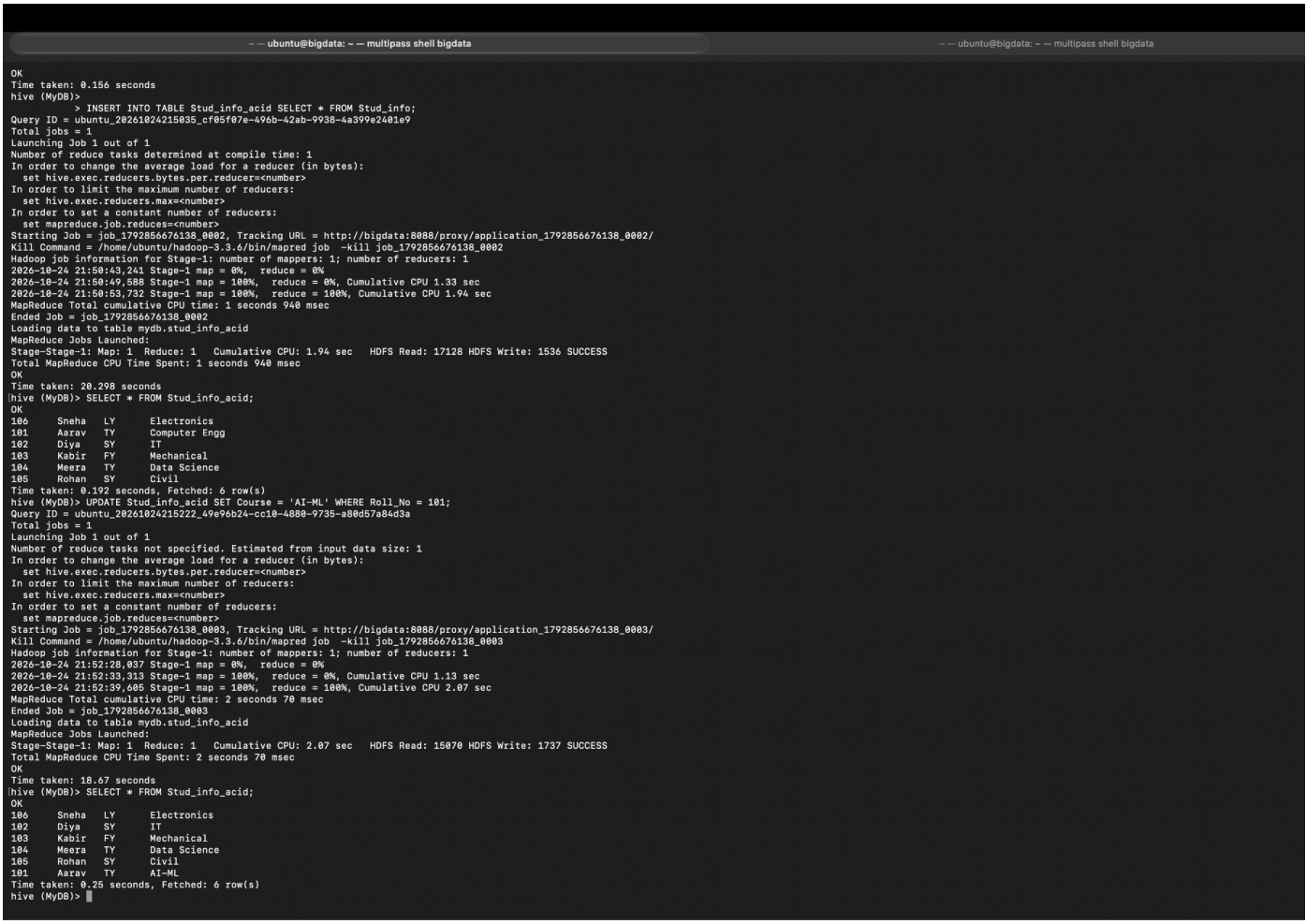
```
~ — ubuntu@bigdata: ~ — multipass shell bigdata                    ~ — ubuntu@bigdata: ~ — multipass shell bigdata

[ubuntu@bigdata:~$ hdfs dfs -mkdir -p /tmp /user/hive/warehouse
[ubuntu@bigdata:~$ hdfs dfs -chmod g+w /tmp
[ubuntu@bigdata:~$ hdfs dfs -chmod g+w /user/hive/warehouse
[ubuntu@bigdata:~$ hdfs dfs -ls /user/hive
Found 1 items
drwxrwxr-x   - ubuntu supergroup          0 2026-10-24 21:14 /user/hive/warehouse
[ubuntu@bigdata:~$ schematool -dbType derby -info
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/ubuntu/apache-hive-3.1.2-bin/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/ubuntu/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Metastore connection URL:        jdbc:derby:;databaseName=/home/ubuntu/metastore_db;create=true
Metastore Connection Driver :    org.apache.derby.jdbc.EmbeddedDriver
Metastore connection User:       APP
Hive distribution version:       3.1.0
Metastore schema version:        3.1.0
schemaTool completed
[ubuntu@bigdata:~$ hive
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/ubuntu/apache-hive-3.1.2-bin/lib/log4j-slf4j-impl-2.10.0.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/ubuntu/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = a6370da5-d928-4535-9bb1-211f9c9129ee

Logging initialized using configuration in jar:file:/home/ubuntu/apache-hive-3.1.2-bin/lib/hive-common-3.1.2.jar!/hive-log4j2.properties Async: true
Hive Session ID = f7e1c072-69e8-4fcf-945a-01950c3d4dbc
Hive-on-MR is deprecated in Hive 2 and may not be available in the future versions. Consider using a different execution engine (i.e. spark, tez) or using Hive 1.X releases.
[hive (default)> show databases;
OK
default
Time taken: 0.315 seconds, Fetched: 1 row(s)
hive (default)> CREATE DATABASE IF NOT EXISTS MyDB;
OK
Time taken: 0.31 seconds
hive (default)> SHOW DATABASES;
OK
default
mydb
Time taken: 0.019 seconds, Fetched: 2 row(s)
[hive (default)> USE MyDB;
OK
Time taken: 0.057 seconds
hive (MyDB)> CREATE TABLE Stud_info (Roll_No INT, Name STRING, Year STRING, Course STRING)
           > ROW FORMAT DELIMITED
           > FIELDS TERMINATED BY ','
           > STORED AS TEXTFILE;
OK
Time taken: 0.417 seconds
hive (MyDB)>
[         > DESCRIBE Stud_info;
OK
roll_no                 int
name                    string
year                    string
course                  string
Time taken: 0.148 seconds, Fetched: 4 row(s)
hive (MyDB)> LOAD DATA INPATH '/info/info.csv' INTO TABLE Stud_info;
Loading data to table mydb.stud_info
OK
Time taken: 0.417 seconds
[hive (MyDB)> SELECT * FROM Stud_info;
OK
101     Aarav   TY      Computer Engg
102     Diya    SY      IT
103     Kabir   FY      Mechanical
104     Meera   TY      Data Science
105     Rohan   SY      Civil
Time taken: 0.834 seconds, Fetched: 5 row(s)
hive (MyDB)> INSERT INTO TABLE Stud_info VALUES (106, 'Sneha', 'LY', 'Electronics');
Query ID = ubuntu_20261024214713_18f6fe7a-1b98-404c-b8e3-9045f970e0f4
```

```
~ — ubuntu@bigdata: ~ — multipass shell bigdata                    ~ — ubuntu@bigdata: ~ — multipass shell bigdata

hive (MyDB)> LOAD DATA INPATH '/info/info.csv' INTO TABLE Stud_info;
Loading data to table mydb.stud_info
OK
Time taken: 0.417 seconds
[hive (MyDB)> SELECT * FROM Stud_info;
OK
101     Aarav   TY      Computer Engg
102     Diya    SY      IT
103     Kabir   FY      Mechanical
104     Meera   TY      Data Science
105     Rohan   SY      Civil
Time taken: 0.834 seconds, Fetched: 5 row(s)
hive (MyDB)> INSERT INTO TABLE Stud_info VALUES (106, 'Sneha', 'LY', 'Electronics');
Query ID = ubuntu_20261024214713_18f6fe7a-1b98-404c-b8e3-9045f970e0f4
Total jobs = 3
Launching Job 1 out of 3
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1792856676138_0001, Tracking URL = http://bigdata:8088/proxy/application_1792856676138_0001/
Kill Command = /home/ubuntu/hadoop-3.3.6/bin/mapred job  -kill job_1792856676138_0001
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
2026-10-24 21:47:27,533 Stage-1 map = 0%,  reduce = 0%
2026-10-24 21:47:32,853 Stage-1 map = 100%,  reduce = 0%, Cumulative CPU 1.81 sec
2026-10-24 21:47:37,024 Stage-1 map = 100%,  reduce = 100%, Cumulative CPU 2.46 sec
MapReduce Total cumulative CPU time: 2 seconds 460 msec
Ended Job = job_1792856676138_0001
Stage-4 is selected by condition resolver.
Stage-3 is filtered out by condition resolver.
Stage-5 is filtered out by condition resolver.
Moving data to directory hdfs://localhost:9000/user/hive/warehouse/mydb.db/stud_info/.hive-staging_hive_2026-10-24_21-47-13_816_3189532594194598222-1/-ext-10000
Loading data to table mydb.stud_info
MapReduce Jobs Launched:
Stage-Stage-1: Map: 1  Reduce: 1   Cumulative CPU: 2.46 sec   HDFS Read: 17647 HDFS Write: 333 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 460 msec
OK
Time taken: 25.871 seconds
[hive (MyDB)> SELECT * FROM Stud_info;
OK
106     Sneha   LY      Electronics
101     Aarav   TY      Computer Engg
102     Diya    SY      IT
103     Kabir   FY      Mechanical
104     Meera   TY      Data Science
105     Rohan   SY      Civil
Time taken: 0.302 seconds, Fetched: 6 row(s)
[hive (MyDB)> UPDATE Stud_info SET Course = 'AI-ML' WHERE Roll_No = 101;
FAILED: SemanticException [Error 10297]: Attempt to do update or delete on table MyDB.Stud_info that is not transactional
hive (MyDB)> CREATE TABLE Stud_info_acid (Roll_No INT, Name STRING, Year STRING, Course STRING)
           > STORED AS ORC
           > TBLPROPERTIES ('transactional'='true');
OK
Time taken: 0.156 seconds
hive (MyDB)>
           > INSERT INTO TABLE Stud_info_acid SELECT * FROM Stud_info;
Query ID = ubuntu_20261024215035_cf05f07e-496b-42ab-9938-4a399e2401e9
Total jobs = 1
Launching Job 1 out of 1
Number of reduce tasks determined at compile time: 1
In order to change the average load for a reducer (in bytes):
  set hive.exec.reducers.bytes.per.reducer=<number>
In order to limit the maximum number of reducers:
  set hive.exec.reducers.max=<number>
In order to set a constant number of reducers:
  set mapreduce.job.reduces=<number>
Starting Job = job_1792856676138_0002, Tracking URL = http://bigdata:8088/proxy/application_1792856676138_0002/
Kill Command = /home/ubuntu/hadoop-3.3.6/bin/mapred job  -kill job_1792856676138_0002
Hadoop job information for Stage-1: number of mappers: 1; number of reducers: 1
```



Handwritten Questions

1. Write the HiveQL commands to create a database named CollegeDB, select the database, and create a table named Student with attributes RollNo, Name, Department, and Marks.
2. Write HiveQL commands to insert at least five student records into the Student table and display all records using a SELECT query.
3. Write HiveQL queries to retrieve students who have scored more than 75 marks and display the results in descending order of marks.
4. Explain the difference between managed and external tables in Hive. Write the commands to display table metadata and describe how to update a record in a Hive table.

MIT SCHOOL OF COMPUTING
Rajbaug, Loni-Kalbhor, Pune

MIT-ADT UNIVERSITY
PUNE, INDIA

BDTL Experiment-6 Hive QL

Q1 Create Database and Student table.

⇒
```
CREATE DATABASE CollegeDB;
USE CollegeDB;

CREATE TABLE STUDENT (
  RollNo INT,
  Name STRING,
  Department String,
  Marks INT,
);
```
This creates college DB and Student table with mentioned four attributes.

Q2 Insert five records and display all.

⇒
```
INSERT INTO TABLE STUDENT VALUES
(101, 'Amit', 'CSE', 85),
(102, 'Priya', 'IT', 92),
(103, 'RAHUL', 'ECE', 68),
(104, 'Sneha', 'CSE', 78),
(105, 'Rohan', 'IT', 72);

SELECT * FROM Student;
```

Inserts adds five records while Select * displays all column and records.

3. Students Scoring More than 75.

⇒
```
SELECT *
FROM Student
WHERE Marks > 75
ORDER BY MARKS DESC;
```

Where filters students scoring above 75 & order by marks DESC sorts them from highest to lowest.

4. Managed vs External Tables

⇒ • Mangged Tables :- Hive manages both Metadata & Data. Dropping it generally deletes both.
• External Table :- Hive manages metadata but the data is stored externally. Dropping it generally preserves data.

```
Describe STudent;
DESCRIBE Formatted Student;
```

Describe shows columns and data types
Describe formatted displays additional table details.

Updates a record requires ACID transaction
support.

```
UPDATE Student;
SET Marks = 90
Where RollNo = 101;
```

This changes the marks of student 101 to 90 if
transactional updates are supported using
an appropriate INSERT OVERWRITE
approach.

Acceptance Criteria

1 The student successfully creates a Hive database and table with the specified schema.

2 The student successfully inserts records and retrieves the stored data using appropriate HiveQL commands.

3 The student executes filtering and sorting queries correctly and obtains the expected results.

4 The student demonstrates an understanding of Hive metadata and managed versus external tables, and correctly explains or executes update operations according to the configured Hive environment.

Conclusion:

The experiment focused on installing Hive and conducting SQL-like operations. It showcased Hive's integration with Hadoop for efficient data processing and analysis.